NFL Database Project

Nicholas Wichman
nswichma@buffalo.edu

Faiyaz Rafee
faiyazra@buffalo.edu

I. PROBLEM STATEMENT

This project aims to create a well-rounded NFL database that allows for a large variety of statistical comparisons between players, coaches, general managers and owners. Currently getting this information requires directly going out and searching for individual statistics and comparing them manually where we wish to make the comparisons automatically through the use a relation database.

A. *Why Not Excel?*

Using excel would still require manually looking through multiple sheets and comparing them manually. Additionally updating relations would require manually changing multiple sheets, which is prone to human error especially when dealing with large datasets such as ours. Finally, using a database allows us to ensure there are no duplicate records along with other constraints we wish to add.

B. *Background*

As both of us are stat nerds when it comes to the NFL the idea of a database implementation to allow us, and others, to find and compare information about various statistics across the league

II. Target User

The target users of this database are primarily NFL fans, analysts and sport researchers who may want to compare teams, players , coaches, managers, owners and stadium information in an efficient way without having to do pain-staking manual searches across an awful number of webpages or excel files. Instead they will easily be able to use basic SQL queries to find connected information, such as being able to compare offensive and defensive production across players or identify which team a player belongs to, and figure out the coach and general manager of said team in the process, and be able to connect the resulting information to the team's stadium. This makes the database very useful for any form of comparison and analysis.

The database would be administered by the developers, which is us in this case, who would be responsible for maintaining the schema, updating the data as needed, and ensuring the integrity of said data between relations. In a real-world use case though, there would be a team with proper connections to the NFL datasets so that it could be updated and maintained in real time thus any interested party like say a sports media group or a content creator would be able to find up to date information easily, without having to have much technical knowledge. The admins would manage the data imports and updates so that the end users can focus on their queries to answer any questions and find the patterns they were looking for that could be very tedious to find otherwise, making it so NFL information becomes much easier to access, compare, and analyze through a full relational database.

III. Relation Schema

Entity	Primary Keys	Foreign Keys
Teams	TeamID	StadiumID CoachID GMID
Coaches	CoachID	TeamID TeamName
Players	PlayerID	TeamID TeamName
Offensive Stats	OffensiveID	PlayerID PlayerName
Defensive Stats	DefensiveID	PlayerID PlayerName
General Managers	GMID	TeamID TeamName
Stadiums	StadiumID	TeamID TeamName CityName
Owners	OwnerID	TeamId TeamName CityName

A. Teams

Name	Type	Notes
TeamID	INT	Primary Key
TeamName	VARCHAR	N/A
City	VARCHAR	N/A
StadiumID	INT	Foreign Key from Stadiums
CoachID	INT	Foreign Key from Coachs
GMID	INT	Foreign Key from GeneralMangers

B. Coaches

Name	Type	Notes
CoachID	INT	Primary Key
CoachName	VARCHAR	N/A
TeamID	INT	Foreign Key from Teams
TeamName	VARCHAR	Foreign Key from Teams
Salary	INT	N/A
YearHired	INT	N/A

C. Players

Name	Type	Notes
PlayerID	INT	Primary Key
Position	VARCHAR	N/A
PlayerName	VARCHAR	N/A
TeamID	INT	Foreign Key from Teams
Salary	INT	N/A
YearJoined	INT	N/A

D. *Offensive Stats*

Name	Type	Notes
OffensiveID	INT	Primary Key
PlayerID	INT	Foreign Key from Players
PlayerName	VARCHAR	Foreign Key from Players
PassYards	INT	N/A
PassTDs	INT	N/A
RushYards	INT	N/A
RushTDs	INT	N/A
ReceivingYards	INT	N/A
ReceivingTDs	INT	N/A

E. *Defensive Stats*

Name	Type	Notes
DefensiveID	INT	Primary Key
PlayerID	INT	Foreign Key from Players
PlayerName	VARCHAR	Foreign Key from Players
Tackles	INT	N/A
Sacks	INT	N/A
Interceptions	INT	N/A
TDs	INT	N/A
TacklesForLoss	INT	N/A
GamesPlayed	INT	N/A

F. *General Managers*

Name	Type	Notes
GMID	INT	Primary Key
Name	VARCHAR	N/A
TeamID	INT	Foreign Key from Teams
TeamName	VARCHAR	Foreign Key from Teams
Salary	INT	N/A
YearHired	INT	N/A

G. *Stadiums*

Name	Type	Notes
StadiumID	INT	Primary Key
TeamName	VARCHAR	Foreign Key from Teams
City	VARCHAR	Foreign Key from Teams
StadiumName	VARCHAR	N/A
YearBuilt	INT	N/A
Cost	INT	N/A

H. *Owners*

Name	Type	Notes
OwnerID	INT	Primary Key
TeamID	INT	Foreign Key from Teams
TeamName	VARCHAR	Foreign Key from Teams
Name	VARCHAR	N/A
NetWorth	INT	N/A
YearBought	INT	N/A

IV. Attribute Information

In this section we will cover default values and nullability of each attribute along with foreign key deletion impacts for each table.

A. Teams

There are no default values any of the attributes in this entity as TeamID is a Primary Key for the entity and StadiumID, CoachID and GMID are all Foreign Keys that are Primary Keys in their respective entities. TeamName and City also should not have a default value as there is no situation where these should be unknown.

Upon deletion of the foreign key attributes the following changes will be made: StadiumID -> Set Null, CoachID -> Set Null and GMID -> Set Null.

B. Coaches

There are no default values for CoachID, CoachName, TeamName and TeamID. Salary will have a default value of 0 and YearHired will have a default value of the current year.

The foreign keys TeamName and TeamID should never be deleted so if they are set to null.

C. *Players*

There are no default values for PlayerID, PlayerName and TeamID. Postion and salary have default values of null and YearJoined has a default value of the current year.

Upon deletion TeamID should be set to null.

D. Offensive Stats

There are no default values for OffensiveID, PlayerID and PlayerName. The rest of the attributes have a default value of 0.

Upon deletion of PlayerID or PlayerName the tuple should be deleted.

E. Defensive Stats

There are no default values for DefensiveID, PlayerID and PlayerName. The rest of the attributes have a default value of 0.

Upon deletion of PlayerID or PlayerName the tuple should be deleted.

F. General Manager

There are no default values for GMID, TeamID, Name and TeamName. Salary will have a default value of 0 and YearHired will have a defulat value of the current year.

The foreign keys TeamName and TeamID should never be deleted so if they are set to null.

G. Stadiums

There are no default values for StadiumID, TeamName and City. StadiumName has a default value of null, YearBuilt has a default value of the current year and Cost has a default value of 0.

The foreign keys TeamName and City should never be deleted so if they are set to null.

H. Owners

There are no default values for OwnerID, TeamID, Name and TeamName. NetWorth has a default value of 0 and YearBought has a default value of the current year.

The foreign keys TeamName and TeamID should never be deleted so if they are set to null.

V. BCNF Verification

In this section we will cover each table adhering to the BCNF standard

A. Teams

FD: TeamID -> TeamName, City, StadiumId, CoachID, GMID

Super Key: TeamName

Since the left side of the functional dependency is a super key this table adheres to BCNF

B. Coaches

FD: CoachID -> CoachName, TeamID, TeamName, Salary, YearHired

Super Key: CoachID

Since the left side of the functional dependency is a super key this table adheres to BCNF

C. *Players*

FD: PlayerID -> Positon, PlayerName, TeamID, Salary, YearJoined

Super Key: PlayerID

Since the left side of the functional dependency is a super key this table adheres to BCNF

D. Offensive Stats

FD: OffensiveID -> PlayerID, PlayerName, PassYards, PassTDs, RushYards, RushTDs, RecievingYards, RecievingTDs

Super Key: Offensive

Since the left side of the functional dependency is a super key this table adheres to BCNF

E. Defensive Stats

FD: DefensiveID -> PlayerID, PlayerName, Tackles, Sacks, Interceptions, TDs, TacklesForLoss, GamesPlayed

Super Key: DefensiveID

Since the left side of the functional dependency is a super key this table adheres to BCNF

F. General Manager

FD: GMID -> Name, TeamID, TeamName, Salary, YearHired

Super Key: GMID

Since the left side of the functional dependency is a super key this table adheres to BCNF

G. Stadiums

FD: StadiumID -> TeamName, City, StadiumName, YearBuilt, Cost

Super Key: StadiumID

Since the left side of the functional dependency is a super key this table adheres to BCNF

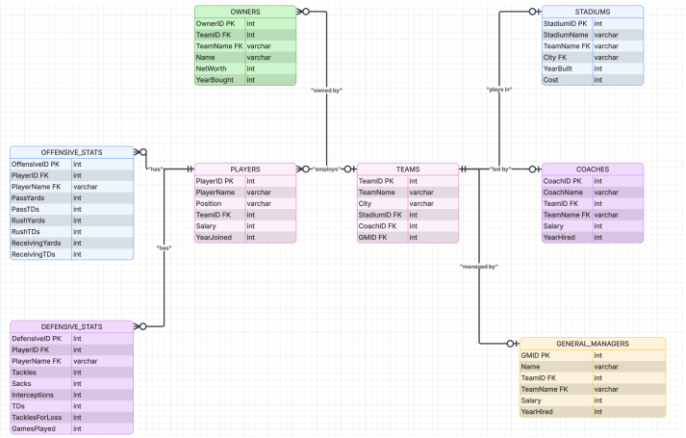
H. Owners

FD: Owner -> OwnerID, TeamName, Name, NetWorth, YearBought

Super Key: OwnerID

Since the left side of the functional dependency is a super key this table adheres to BCNF

VI. ER Diagram



VII. Relations

In this section we will outline the relationships between our entity sets along with their type.

A. One-To-One relationships
- Team to Coaches: Only one coach per team and a coach can only coach one team.
- Team to Owner: Only one owner can own a team and only one team can be owned by a single owner.
- Team to Stadiums: Teams only have one home stadium, and stadiums only have one home team
- Team to General Managers: Teams can only have one general manager, and a general manager can only be employed by one team.
- Players to Offensive Stats: Each player has only one offensive stats record associated with them.
- Players to Defensive Stats: Each player has only one defensive stats record associated with them.

B. One-To-Many Relationship
- Team to Players: One team can employ many players and players can only play for one team.

VIII. Database

In order to better fit our schema we will be generating our data through an AI model that will get the information from Pro Football Reference. We will download the data into CSV files and then add a few aspects like identification numbers starting from one.